TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT NO. 3

Playfair Cipher over the Romanian Alphabet

C++ implementation with a 5×6 matrix and Unicode text processing

Student: Gabriel Balan

Group: FAF-241

Coordinator: asist. univ. Zaica Maia

Chișinău - 2026

Contents

Abstract **3**

Introduction **4**

1 Short theory **5**

1.1 Playfair and the Romanian matrix 5

1.2 Preparing the plaintext 6

1.3 Encryption and decryption rules 6

2 Implementation **7**

2.1 Text representation and validation 7

2.2 Constructing the matrix 8

2.3 Forming plaintext digraphs 9

2.4 Transforming the pairs 10

2.5 Menu and formatted output 11

3 Results **12**

3.1 Encryption of Romanian text 12

3.2 Decryption and filler interpretation 13

3.3 Boundary rules and repeated letters 14

3.4 Input validation and supplementary checks 15

Conclusions **16**

Source code **17**

ABSTRACT

This laboratory implements encryption and decryption using the Playfair cipher for Romanian text. The original 31-letter alphabet is reduced to 30 symbols by replacing J with I, allowing all remaining letters to fit into a rectangular matrix with five rows and six columns. A keyword of at least seven letters determines the matrix ordering. The program normalizes input, forms plaintext digraphs, inserts filler letters when needed, and applies the same-row, same-column, or rectangle rule to each pair. Decryption reverses the shifts while preserving pair boundaries. Custom UTF-8 and UTF-32 conversion functions support Romanian diacritics, and the interactive menu displays the matrix, formatted input, full result, and formatted result. The report describes the actual C++ implementation and presents executable results, boundary examples, and round-trip verification.

Keywords: Playfair cipher, Romanian alphabet, digraphs, rectangular matrix, C++, UTF-8, UTF-32

INTRODUCTION

The objective of Laboratory Work No. 3 is to study substitution methods in which encryption depends on more than the identity of an isolated plaintext letter. The supplied handout presents Vigenère and Playfair; this report covers Task 3.1, the Playfair implementation [1]. The task requires a Romanian alphabet, a key containing at least seven letters, an operation selector, and the ability to enter either plaintext or ciphertext.

Unlike the Caesar cipher studied in Laboratory Work No. 1, Playfair processes two letters at a time. Their positions in a keyword-derived matrix determine the replacement pair. A letter can therefore have different encrypted forms depending on its partner. The implementation also retains the custom Unicode conversion approach used in the earlier laboratory, so a Romanian letter is processed as one code point even when it occupies multiple UTF-8 bytes.

The application is contained in `Lab_3/src/main.cpp`, with CMake requesting C++23 and strict compiler warnings [2]. The interface repeats until the user chooses exit. Both directions are available through the same menu, and the formatted pair output makes each transformation visible. Restoring word boundaries and interpreting inserted filler letters remain manual operations, as intended by the laboratory.

1 SHORT THEORY

1.1 Playfair and the Romanian matrix

Playfair is a digraph substitution cipher: its basic unit is an ordered pair of letters. The handout describes a conventional English 5×5 matrix and permits other dimensions when the alphabet has a different size [1]. This implementation begins with the Romanian alphabet:

AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ.

J is replaced by I in both messages and keys. The resulting 30 symbols fit exactly into a 5×6 matrix. Romanian diacritics remain separate letters: Ă and Â are distinct from A, Î is distinct from I, and Ș and Ț retain their own cells. No additional symbols or empty cells are needed.

The matrix is filled row by row. First, the distinct letters of the normalized key are inserted in their first-occurrence order. All unused letters of the alphabet follow in alphabetical order, excluding J. The minimum key length is checked before duplicate removal; seven letters need not be seven distinct letters.

For example, the key CRIPTOGRAFIE contributes CRIPTOGAFE. The repeated R and I do not create additional cells. The completed matrix is:

C	R	I	P	T	O
G	A	F	E	Ă	Â
B	D	H	Î	K	L
M	N	Q	S	Ș	Ț
U	V	W	X	Y	Z

Figure 1.1 – Playfair matrix generated from the key CRIPTOGRAFIE.

The matrix in Figure 1.1 is used for both encryption and decryption.

1.2 Preparing the plaintext

Plaintext is uppercased, J is merged with I, cedilla variants are normalized, and ordinary spaces are removed. Romanian diacritics are retained; punctuation and digits are rejected.

Pairs are read from left to right. Different letters are consumed together. Identical letters are separated with Q, or X when the repeated letter is Q, leaving the second letter for the next pair. A final unpaired letter receives the same filler. Table 1.1 shows three examples:

Normalized text	Prepared pairs
BALLOON	BA LQ LO ON
ANA	AN AQ
QQQ	QX QX QX

Table 1.1 – Plaintext preparation with the Q/X filler convention.

Identical letters across a pair boundary are allowed. The algorithm consumes one letter after inserting a filler and two after an ordinary pair.

1.3 Encryption and decryption rules

Let the letters of a pair occupy positions (r_1, c_1) and (r_2, c_2), using row indices $0, \ldots, 4$ and column indices $0, \ldots, 5$. Denote the matrix by M. Encryption uses the following rules:

1. If $r_1 = r_2$, move both letters one column right, wrapping within the row: $c_i' = (c_i + 1) \bmod 6$.
2. Otherwise, if $c_1 = c_2$, move both letters one row down, wrapping within the column: $r_i' = (r_i + 1) \bmod 5$.
3. Otherwise, exchange the columns: the output is $M[r_1][c_2]$ followed by $M[r_2][c_1]$.

For decryption, rightward movement becomes leftward movement, and downward movement becomes upward movement:

$$c_i' = (c_i + 5) \bmod 6, \qquad r_i' = (r_i + 4) \bmod 5. \tag{1.1}$$

Adding 5 or 4 produces the inverse displacement without a negative intermediate value. The rectangle rule is its own inverse, since exchanging the two columns twice restores the original positions.

Ciphertext is paired directly, without filler insertion, and must have even length. Decryption retains fillers; capitalization, spaces, and the I/J distinction are lost during preprocessing.

2 IMPLEMENTATION

2.1 Text representation and validation

Terminal input and output use UTF-8 byte strings. The custom decoder converts them to UTF-32 code points, so each Romanian letter occupies one element during key validation, pairing, and matrix lookup. It checks byte-sequence structure and scalar-value ranges; the encoder converts results back to UTF-8.

The normalization stage merges J with I and uses corresponding uppercase/lowercase alphabet positions:

C++ SOURCE main.cpp: 228–243

```
228         //extra we add here for Playfair
229         else if (c == U'J')
230             c = U'I';
231         else if (c == U'j')
232             c = U'i';
233 
234         // Convert lowercase Romanian alphabet to uppercase
235         // using our alphabet ordering.
236 
237         auto pos{g_ralph_lower.find(c)};
238 
239         if (pos != std::u32string_view::npos)
240             c = g_ralph[pos];
241 
242     }
243     std::erase(str, emptySpace);
```

Listing 2.1. Playfair normalization and ordinary-space removal.

Ordinary spaces are erased after traversal. Cedilla variants of Ș and Ț are normalized earlier in the same function. Unsupported characters produce a diagnostic and another message prompt.

Key length is measured after normalization in UTF-32. Repeated letters count toward the seven-letter minimum; spaces do not. A short key is requested again:

C++ SOURCE main.cpp: 379–385

```
379         std::u32string key{};
380         do
381         {
382             key = utf8_to_utf32(getMessage("Insert your key (at least 7 letters): "));
383             if (key.size() < 7)
384                 std::cout << "The key must contain at least 7 letters.\n";
385         } while (key.size() < 7);
```

Listing 2.2. Checking the normalized key length.

2.2 Constructing the matrix

The type `Matrix` is a nested `std::array` with five rows of six `char32_t` values. Matrix construction concatenates the key with the full Romanian alphabet, skips J, and appends each other letter only at its first occurrence:

C++ SOURCE main.cpp: 311–326

```
311 using Matrix = std::array<std::array<char32_t, 6>, 5>;
312
313 Matrix makeMatrix(std::u32string_view key)
314 {
315     std::u32string letters{};
316     for (char32_t c : std::u32string(key) + std::u32string(g_ralph))
317     {
318         if (c != U'J' && letters.find(c) == std::u32string::npos)
319             letters += c;
320     }
321
322     Matrix matrix{};
323     for (size_t i{}; i < letters.size(); ++i)
324         matrix[i / 6][i % 6] = letters[i];
325     return matrix;
326 }
```

Listing 2.3. Constructing the keyword-derived 5 × 6 matrix.

Because every permitted normalized key letter belongs to the reduced alphabet, the collected sequence has exactly 30 elements. Integer division by 6 selects the row, and the remainder selects the column. The fixed array dimensions reflect the rectangular layout directly.

The `findPosition` helper scans the matrix and returns a `std::pair<size_t, size_t>` containing row and column indices. A complete scan examines at most 30 cells. The normal call path guarantees that each input letter is present because validation and normalization precede transformation.

2.3 Forming plaintext digraphs

The alias `Formatted` represents a vector of pairs of Unicode code points. `formatMessage` converts the normalized input to UTF-32, reserves output capacity, and walks through the letters:

C++ SOURCE main.cpp: 294–308

```
294     size_t i{};
295     for(; i + 1 < mlen; ++i) // Only process letters that have a next element.
296     {
297         char32_t first{message[i]};
298         bool same{first == message[i+1]};
299         char32_t second{same ? (first == g_luChar ? U'X' : g_luChar) : message[i+1]};
300 
301         output.emplace_back(first, second);
302 
303         if (!same) ++i; //if its not the same, no extra char added so 2 have been
            ↪  processed.
304 
305     }
306     if (i < mlen)
307         output.emplace_back(message[i], message[i] == g_luChar ? U'X' : g_luChar);
308     return output;
```

Listing 2.4. Preparing plaintext pairs and inserting Q or X.

The loop condition checks that a next element exists before reading it. On an ordinary pair, the extra increment inside the loop consumes the second letter. On a repeated pair, only the first letter is consumed, leaving the second for the next iteration. The final check handles an odd number of remaining letters. Using X when the first letter is Q avoids creating an identical filler pair such as QQ.

This function is used only for encryption. Applying it to ciphertext would insert or shift letters and break the correspondence between encrypted pairs and matrix operations. The decryption branch instead checks even length and constructs the pairs directly from adjacent elements.

2.4 Transforming the pairs

The function `playfair` accepts the matrix, a vector of pairs, and a Boolean selecting decryption. It returns a UTF-32 string. The inverse shifts are represented by positive unsigned values:

C++ SOURCE main.cpp: 337–362

```
std::u32string playfair(const Matrix& matrix, const Formatted& message, bool decrypt)
{
    std::u32string output{};
    size_t rowShift{decrypt ? 4u : 1u};
    size_t colShift{decrypt ? 5u : 1u};
    for (const auto& [first, second] : message)
    {
        auto [r1, c1] = findPosition(matrix, first);
        auto [r2, c2] = findPosition(matrix, second);
        if (r1 == r2)
        {
            c1 = (c1 + colShift) % 6;
            c2 = (c2 + colShift) % 6;
        }
        else if (c1 == c2)
        {
            r1 = (r1 + rowShift) % 5;
            r2 = (r2 + rowShift) % 5;
        }
        else
            std::swap(c1, c2);

        output += matrix[r1][c1];
        output += matrix[r2][c2];
    }
    return output;
```

Listing 2.5. Applying the forward and inverse Playfair rules.

Each iteration obtains two positions, handles the same-row and same-column cases, or swaps the columns for a rectangle. The output letters are appended in the original pair order. Row arithmetic uses modulo 5 and column arithmetic uses modulo 6; using one shared dimension would produce incorrect results for this matrix.

For a message of n letters, the fixed-size position scans give linear processing time, $O(n)$. Inserting fillers can at most double the plaintext length, and the strings and pair vector require $O(n)$ storage. The implementation favors short, explicit operations over an additional lookup structure for only 30 letters.

2.5 Menu and formatted output

The main loop offers `1. Encrypt`, `2. Decrypt`, and `0. Exit`. An invalid choice returns to the menu. A valid operation reads the key, builds and displays the matrix, and requests a message. An odd-length ciphertext prints a diagnostic and returns to the menu; invalid characters are handled by the message retry loop.

Both operations display the input as digraphs, the result as one string, and the result as digraphs. Result formatting uses existing two-letter slices:

C++ SOURCE main.cpp: 413–424

```
413         std::cout << "Formatted input: ";
414         for (const auto& [first, second] : styled)
415             std::cout << utf32_to_utf8(std::u32string{first, second}) << ' ';
416         std::cout << '\n';
417 
418         std::u32string result{playfair(matrix, styled, decrypt)};
419         std::cout << (decrypt ? "Decrypted: " : "Encrypted: ")
420                   << utf32_to_utf8(result) << '\n';
421         std::cout << (decrypt ? "Formatted decrypted: " : "Formatted encrypted: ");
422         for (size_t i{}; i < result.size(); i += 2)
423             std::cout << utf32_to_utf8(result.substr(i, 2)) << ' ';
424         std::cout << '\n';
```

Listing 2.6. Displaying input pairs, the result string, and result pairs.

It does not call the plaintext preparation function again. This preserves ciphertext boundaries and retains any filler letters in the decrypted output. The user can compare each input pair directly with the pair in the corresponding result position. End-of-input ends the application cleanly when reading a menu choice or message.

The numeric-key helpers retained near the beginning of the source are unused by this Playfair interface. Playfair uses a textual keyword and does not request the Caesar shift from the earlier laboratory.

3 RESULTS

3.1 Encryption of Romanian text

The following example uses the key `CRIPTOGRAFIE` and the same Romanian sentence used in Laboratory Work No. 1:

Trimite două sute de mii de soldați să cucerească Moldova

After normalization the plaintext contains 48 letters:

NORMALIZED INPUT

```
TRIMITEDOUĂSUTEDEMIIDESOLDAȚISĂCUCEREASCĂMOLDOVA
```

Figure 3.1 – Normalized Romanian plaintext before pair preparation.

Preparing the pairs inserts Q between the two I letters and adds a final Q, producing 50 letters. The excerpt below was captured from the compiled executable. Repeated menu prompts are omitted; matrix and result values are preserved.

PROGRAM OUTPUT · ENCRYPTION

```
Matrix:
C R I P T O
G A F E Ă Â
B D H Î K L
M N Q S Ș Ț
U V W X Y Z
Formatted input: TR IM IT ED OU ĂS UT ED EM IQ ID ES OL DA ȚI SĂ CU CE RE AS CĂ MO LD OV AQ
Encrypted: OICQPOAÎCZEȘYCAÎGSFWRHÎXÂȚNDQOȘEGCPGPAENTGȚCBHRZFN
Formatted encrypted: OI CQ PO AÎ CZ EȘ YC AÎ GS FW RH ÎX ÂȚ ND QO ȘE GC PG PA EN TG ȚC BH
↪  RZ FN
```

Figure 3.2 – Encryption with key CRIPTOGRAFIE: matrix, prepared plaintext, and ciphertext.

For the first pair, T and R occupy the first row in columns 4 and 1. Moving right produces O and I. The pair IM forms a rectangle: I is in row 0, column 2, while M is in row 3, column 0. Exchanging columns gives C and Q. These explain the initial encrypted pairs `OI CQ`.

3.2 Decryption and filler interpretation

Selecting operation 2 and entering the ciphertext with the same key produces:

PROGRAM OUTPUT · DECRYPTION

```
Matrix:
C R I P T O
G A F E Ă Â
B D H Î K L
M N Q S Ș Ț
U V W X Y Z
Formatted input: OI CQ PO AÎ CZ EȘ YC AÎ GS FW RH ÎX ÂȚ ND QO ȘE GC PG PA EN TG ȚC BH RZ FN
Decrypted: TRIMITEDOUĂSUTEDEMIQIDESOLDAȚISĂCUCEREASCĂMOLDOVAQ
Formatted decrypted: TR IM IT ED OU ĂS UT ED EM IQ ID ES OL DA ȚI SĂ CU CE RE AS CĂ MO LD
↪  OV AQ
```

Figure 3.3 – Decryption with the same key preserves the prepared plaintext pairs.

The output is exactly the prepared plaintext used during encryption. The inserted Q in `MIQI` separates the repeated I letters from `MII`, and the final Q pads the last A. For this particular sentence, removing those two known fillers restores the normalized original message. The human-readable sentence can then be reconstructed by restoring word boundaries.

The program deliberately leaves this interpretation to the reader. Q can be a real letter, and an internal Q between equal letters need not always have been inserted. Removing every Q, or every apparent filler pattern, would not be a generally reversible operation. The printed pairs preserve the information actually produced by Playfair without claiming to recover the original spacing or I/J distinction.

3.3 Boundary rules and repeated letters

A separate key, AĂÂBCDE, produces the alphabet-ordered matrix. Its first row is A Ă Â B C D, and its final row is U V W X Y Z. This makes wrapping particularly easy to verify. The following cases were executed against the program:

Input	Encrypted	Explanation
DA	AĂ	Same row; D wraps to A and A moves to Ă.
UA	AE	Same column; U wraps to A and A moves to E.
AF	ĂE	Rectangle; exchange columns while retaining rows.

Table 3.1 – Verified matrix rules with key AĂÂBCDE.

Decrypting each result recovers its input pair. Additional examples verify preparation: BALLOON becomes BA LQ LO ON, and QQQ becomes QX QX QX. In both cases decryption reproduces the prepared pairs, including their fillers. The alternative X filler is necessary for a message consisting of Q letters.

3.4 Input validation and supplementary checks

The application was built with `cmake --build build`. The CMake configuration enables `-Wall`, `-Wextra`, `-Wpedantic`, `-Werror`, and additional conversion and shadowing warnings. The build completed successfully.

Executable checks covered an invalid menu choice, a three-letter key, punctuation in the message, and an odd-length ciphertext. The observed responses were:

VALIDATION OUTPUT

```
Choose 1, 2 or 0.
The key must contain at least 7 letters.
! Is not a valid character! Use A-Z or Romanian letters (ĂÂÎȘȚ), lowercase or uppercase.
Ciphertext must contain an even number of letters.
```

Figure 3.4 – Diagnostics for invalid menu, key, message, and ciphertext input.

The invalid menu choice is retried; a short key is requested again; punctuation causes another message prompt. Odd-length ciphertext returns to the main menu because the inverse operation requires complete pairs.

A temporary execution script also enumerated all $30 \cdot 29 = 870$ ordered pairs of distinct letters. Each pair was encrypted through the menu using `CRIPTOGRAFIE`, then its ciphertext was decrypted with the same key. All 870 results matched the original pairs. The explicit boundary examples above additionally check known outputs, since a round trip alone could miss a mistake shared by both directions. Repeated-letter and Romanian normalization examples were also checked during implementation.

These checks demonstrate the expected transformation and interface behavior for the tested inputs. They were performed against the actual executable; the console excerpts are text transcripts, not screenshots. No permanent test framework or external cryptography library is required by the program.

CONCLUSIONS

The laboratory implements the complete interactive Playfair workflow for Romanian messages. Merging J with I reduces the alphabet to 30 symbols, allowing a fully populated 5×6 matrix. Keyword deduplication preserves first-occurrence order, while pair preparation handles repeated letters and odd-length plaintext using Q or, when necessary, X.

The rectangular layout makes the distinction between row and column arithmetic explicit. Encryption shifts within six columns or five rows, while decryption uses their inverse shifts. The rectangle transformation is shared by both directions. Known boundary examples and 870 distinct-pair round trips confirm the implemented rules, and the menu exposes both operations with formatted pair output.

The custom UTF-8/UTF-32 conversion layer supports Romanian diacritics throughout validation, key length checks, matrix lookup, and terminal output. Encoding and cipher operations remain separate: Unicode determines how a letter is represented, while its matrix coordinates determine how it is encrypted.

Playfair illustrates pair-based substitution but does not provide modern cryptographic security. Repeated digraphs retain structure, and preprocessing loses capitalization, word boundaries, and the I/J distinction. Filler interpretation and restored spacing require human judgment. The exercise therefore develops precise text processing and reversible matrix operations while showing the limits of a classical cipher.

SOURCE CODE

The implementation is stored in `Lab_3/src/main.cpp`, and the build configuration is stored in `Lab_3/CMakeLists.txt`. The laboratory repository is:

`https://github.com/AverageCEnj0yer/Cryptography`

The report uses the same UTM LaTeX class and author metadata as Laboratory Work No. 1. Source excerpts are read directly from the current C++ file during compilation. Captured console excerpts are stored under `report/data/`. To rebuild the application and run it from `Lab_3`:

BUILD AND RUN

```
cmake -S . -B build
cmake --build build
./build/lab3
```

Figure 3.5 – Commands used to build and launch the laboratory application.

To regenerate this report, run `tectonic main.tex` from the `Lab_3/report` directory. The local class, logo, metadata, transcript files, and adjacent source file are required.

BIBLIOGRAPHY

[1] *Lucrare de laborator nr. 3. Cifruri polialfabetice*. Supplied course handout, Sections 3.2.2–3.2.4 and Task 3.1, pp. 3–7.

[2] Gabriel Balan. *Laboratory Work No. 3: Playfair implementation*. Local source `Lab_3/src/main.cpp` and `Lab_3/CMakeLists.txt`, with executable verification performed on 26 October 2026.